

Formal Languages and Compilers (Linguaggi Formali e Compilatori)

prof. Luca Breveglieri
(prof. S. Crespi Reghizzi, prof. A. Morzenti)

Written exam - 18 september 2012 - Part I: Theory

NAME:

SURNAME:

ID:

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
 - I (80%) Theory:
 1. regular expressions and finite automata
 2. free grammars and pushdown automata
 3. syntax analysis and parsing
 4. translation and semantic analysis
 - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the regular expression R below, over the alphabet $\{a, b\}$:

$$R = \left(a a \mid a b \mid b (b b)^* \right)^+$$

Answer the following questions:

- (a) List all the strings of length less than or equal to three generated by the regular expression R , alphabetically ordered with $a < b$. Describe informally (that is by words) and synthetically the regular language $L(R)$.
 - (b) By a method of your choice (systematic or intuitive) find a finite deterministic automaton A that recognises the regular language $L(R)$.
 - (c) (optional) Explain if the regular expression R is ambiguous and in the case it is so, write a regular expression R' equivalent to R but not ambiguous.
-

2 Free Grammars and Pushdown Automata 20%

1. Answer the following questions and adequately justify your answers:

- (a) Consider a language over the alphabet $\{a, b, c\}$, which consists of strings of odd length with a c in the midpoint, such that if in a position on the left of c there is an a (respectively, a b) then in the symmetrical position on the right of c there is a b (respectively, an a), and viceversa. Valid sample strings are below:

$abbc a a b$ $b a b b c a a b a$

Write a grammar for this language and explain if the language can be recognised by a deterministic pushdown automaton.

- (b) Consider a language over the alphabet $\{a, b\}$, which consists of strings of length multiple of four that are sequences of pairs aa and bb , such that if in a position on the left of the midpoint there is an a (respectively, a b) then in the symmetrical position on the right of the midpoint there is a b (respectively, an a), and viceversa. Valid sample strings are below:

$bb a a a a b b b b a a$ $a a b b a a a a b b b b a a b b$

Write a grammar for this language and explain if the language can be recognised by a deterministic pushdown automaton.

- (c) (optional) Consider the following grammar G , not extended (axiom S):

$$G \left\{ \begin{array}{l} S \rightarrow X \mid Y \\ X \rightarrow a Y b \mid a b \\ Y \rightarrow b X a \mid b a \end{array} \right.$$

Explain what the simplest deterministic automaton is that recognises the language generated by grammar G .

2. Consider the samples below of boolean expression of the historic programming language Algol:

$x > z$	relational operator
$y > x \vee z < w$	
$p \wedge q \vee x > y$	
$p \wedge (q \vee x > y)$	brackets are permitted
$g \supset \neg q \wedge b \wedge \neg c \vee d \vee e \supset \neg f$	
if $x < w$ then $y > w$ else $p \vee q$	in if the else branch is mandatory
if if q then b else f then g else $x < z$	construct if is permitted to nest

The symbols x, y, w, z specify arithmetic variables; the other symbols (p, q, g, b, c , etc.) specify boolean variables. The following operators are admitted in the expressions:

relational operators: $> <$ apply to arithmetic variables

logical operators: $\neg \wedge \vee \supset$ (implication) apply to boolean variables and expressions

operator precedence: the computation order of the operators in an expression is *usually* from left to right, but with the following additional precedence rules:

- first (that is max priority): $< >$
- second: $\neg$
- third: $\wedge$
- fourth: $\vee$
- fifth: $\supset$
- sixth: **if** ... **then** ... **else** ...

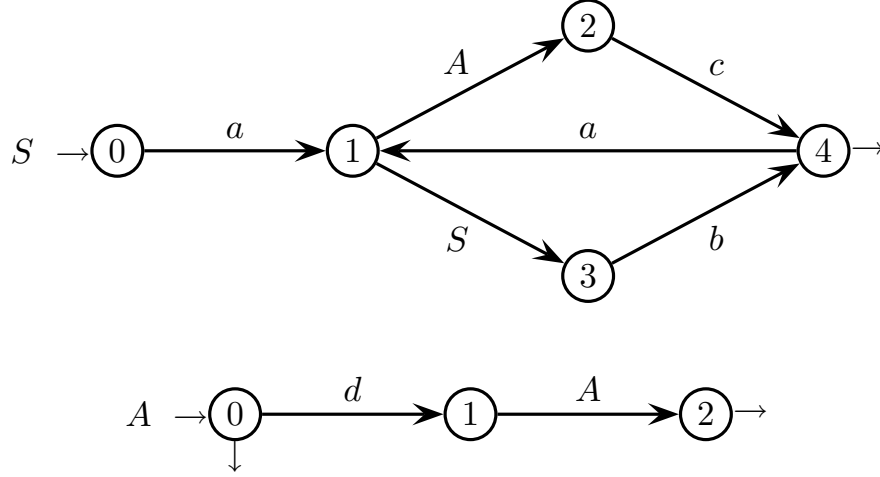
Answer the following questions:

- (a) Write a grammar G , in the BNF form (not extended) and not ambiguous, that generates the language of the boolean expressions described above. Denote the arithmetic variables by the terminal character **a** and the boolean ones by **b**.
- (b) Draw the syntax tree of the sample boolean expression below:

if ($x < w \supset p$) $\vee q$ **then** q **else** $p \wedge q$

3 Syntax Analysis and Parsing 20%

1. Consider the grammar G below, in the extended form, denoted as a network of recursive machines (axiom S), over the terminal alphabet $\{a, b, c, d\}$ and the nonterminal alphabet $\{S, A\}$:



Answer the following questions (by the extended syntax analysis methodology):

- (a) Explain if grammar G is of type $ELR(1)$.
- (b) Explain if grammar G is of type $ELL(1)$.
- (c) (optional) Write the procedure S of the ELL syntax analyser of grammar G , using some kind of pseudo-code.

Notice: if you prefer to answer by the classical LR syntax analysis methodology, please use the grammar below, not extended, which is equivalent to G (axiom S_0):

$$\left\{ \begin{array}{l} S_0 \rightarrow S S_0 \mid S \\ S \rightarrow a A c \mid a S b \\ A \rightarrow d A \mid \varepsilon \end{array} \right.$$

4 Translation and Semantic Analysis 20%

1. Consider the language generated by the regular expression $(a^+c)^+$, the strings of which are sequences of at least one factor of type $a^n c$ with $n \geq 1$. One wants to translate such strings and transform every factor $a^n c$ in one of type $b^k s$ with $k \geq 0$, where it holds $k = \frac{n}{2} - 1$ if n is even or $k = \lceil \frac{n}{2} \rceil$ if n is odd (for n odd the notation $\lceil \frac{n}{2} \rceil$ specifies the smallest integer greater than $\frac{n}{2}$). For example:

$$\tau(a^3 c a^4 c a^5 c a^2 c) = b b s b s b b b s s$$

Answer the following questions:

- (a) Explain if the translation τ is one-to-one.
 - (b) Explain if the translation τ is computable by a finite deterministic transducer.
 - (c) Draw a finite transducer T that computes the translation τ , if possible deterministic.
 - (d) (optional) Define the translation τ by a regular transduction expression R_τ .
-

2. An aircraft flight is described by a 3-uple of the form below:

$$\langle \text{flight} \rangle \rightarrow (\text{flight_number}, \text{destination}, \text{cost})$$

where **flight_number** is an identification tag, **destination** is an airport name and **cost** is a currency value (they can be modeled as terminal). A catalogue of flights is described by a list of such 3-uples, this way:

$$\langle \text{catalogue} \rangle \rightarrow \langle \text{flight} \rangle \langle \text{catalogue} \rangle \mid \varepsilon$$

To select flights in the catalogue, one can write a query described by the syntax below:

$$\langle \text{query} \rangle \rightarrow \textbf{select flight_number from } \langle \text{catalogue} \rangle \textbf{ where } \langle \text{condition} \rangle$$

$$\langle \text{condition} \rangle \rightarrow \textbf{destination} = \text{airport} \mid \dots$$

For example the query below:

```
select flight_number from
( LH3, Madrid, 230 ) ( AZ2, Madrid, 305 ) ( AF3, Paris, 250 )
where destination = Madrid
```

gives this answer:

$$\textit{selection} = \{ LH3, AZ2 \}$$

Answer the following questions:

- (a) Write an attribute grammar G that computes the answer to a query. The syntax support to use is below (axiom $\langle \text{query} \rangle$):

$$\begin{array}{lcl} \langle \text{query} \rangle & \rightarrow & \textbf{select flight_number from } \langle \text{catalogue} \rangle \\ & & \textbf{where } \langle \text{condition} \rangle \\ \hline \langle \text{catalogue} \rangle & \rightarrow & \langle \text{flight} \rangle \langle \text{catalogue} \rangle \\ \hline \langle \text{catalogue} \rangle & \rightarrow & \varepsilon \\ \hline \langle \text{flight} \rangle & \rightarrow & (\text{flight_number}, \text{destination}, \text{cost}) \\ \hline \langle \text{condition} \rangle & \rightarrow & \textbf{destination} = \text{airport} \end{array}$$

The answer to the query must be assigned to the left attribute *selection* associated with the axiom $\langle \text{query} \rangle$. For each attribute chosen, in the table on the next page please write the attribute category (left/right) and which nonterminal or terminal symbols the attribute is associated with.

Write the necessary semantic rules in the table on the next pages and draw the decorated syntax tree of the sample query above along with the attribute values.

- (b) (optional) Draw the functional dependence graphs for every rule and explain if the attributes are computable by the one-sweep method.

<i>cat.</i>	<i>name</i>	<i>(non)terminal</i>	<i>type</i>	<i>meaning</i>
left	<i>selection</i>	query	set of flight_number	answer to the query

Furthermore, there are the attributes (conventionally right) that return the values of the terminal symbols `flight_number`, `destination`, `cost` and `airport`, which have the same names as the terminal symbols they refer to. They are obvious and not listed here.

$\langle \text{query} \rangle_0 \rightarrow$ **select** **flight_number**
from $\langle \text{catalogue} \rangle_1$
where $\langle \text{condition} \rangle_2$

$\langle \text{catalogue} \rangle_0 \rightarrow \langle \text{flight} \rangle_1 \langle \text{catalogue} \rangle_2$

$\langle \text{catalogue} \rangle_0 \rightarrow \varepsilon$

$\langle \text{flight} \rangle_0 \rightarrow (\text{flight_number}_1,$
 $\text{destination}_2,$
 $\text{cost}_3)$

$\langle \text{condition} \rangle_0 \rightarrow \text{destination} = \text{airport}_1$

